COMPUTER VISION

LABSHEET-4

NAME-HARSH RAJ

CU ID- 240251762

```
# Q1. Import libraries and load a grayscale image
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image in grayscale
image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    raise FileNotFoundError("Image not found. Check the file path.")

print("Image shape:", image.shape)

#Q2. Compute the Discrete Fourier Transform (DFT)

# Convert image to float32
image_float = np.float32(image)
# Compute DFT
dft = cv2.dft(image_float, flags=cv2.DFT_COMPLEX_OUTPUT)
# Shift zero frequency component to the center
dft_shift = np.fft.fftshift(dft)
```

```
#Q3. Shift the zero-frequency component

# Calculate magnitude spectrum
magnitude_spectrum = cv2.magnitude(
    dft_shift[:, :, 0],
    dft_shift[:, :, 1]
)
# Log transformation for better visualization
magnitude_spectrum = 20 * np.log(
    magnitude_spectrum + 1
)

#Q4. Display and analyze the magnitude spectrum

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.tight_layout()
```

```
plt.show()

#Q5. Low-Pass Frequency Filter

rows, cols = image.shape
crow, ccol = rows // 2, cols // 2
# Create a low-pass mask

radius = 50
low_pass_mask = np.zeros((rows, cols, 2), np.float32)
y, x = np.ogrid[:rows, :cols]
distance = np.sqrt((x - ccol)**2 + (y - crow)**2)
low_pass_mask[distance <= radius] = 1

# Apply low-pass filter
low_pass_dft = dft_shift * low_pass_mask

#Q6. High-Pass Frequency Filter

# Create high-pass mask
high_pass_mask = np.ones((rows, cols, 2), np.float32)

high_pass_mask[distance <= radius] = 0

# Apply high-pass filter
high_pass_dft = dft_shift * high_pass_mask
```

```
#Q7. Inverse Fourier Transform

# Shift back
low_pass_ishift = np.fft.ifftshift(low_pass_dft)

# Inverse DFT
low_pass_result = cv2.idft(low_pass_ishift)

# Convert complex result to magnitude
low_pass_result = cv2.magnitude(
    low_pass_result[:, :, 0],
    low_pass_result[:, :, 1]
)

# Normalize to 0-255
low_pass_result = cv2.normalize(
    low_pass_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

low_pass_result = np.uint8(low_pass_result)
#High-pass reconstruction
```

```
# Shift back
high_pass_ishift = np.fft.ifftshift(high_pass_dft)

# Inverse DFT
high_pass_result = cv2.idft(high_pass_ishift)

# Convert complex result to magnitude
high_pass_result = cv2.magnitude(
    high_pass_result[:, :, 0],
    high_pass_result[:, :, 1]
)

# Normalize to 0-255
high_pass_result = cv2.normalize(
    high_pass_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

high_pass_result = np.uint8(high_pass_result)

#Q8. Compare the images

plt.figure(figsize=(15, 10))
```

```
plt.subplot(2, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("DFT Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(low_pass_result, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")

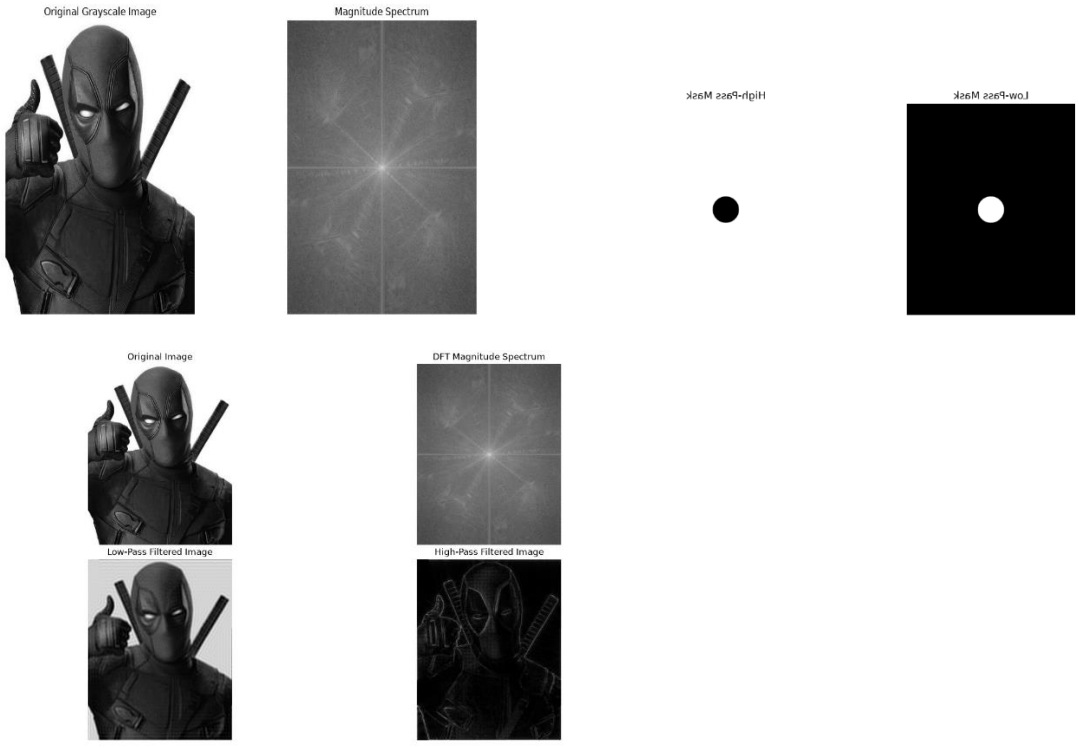
plt.subplot(2, 2, 4)
plt.imshow(high_pass_result, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 4))
```

```
plt.subplot(1, 2, 1)
plt.imshow(low_pass_mask[:, :, 0], cmap="gray")
plt.title("Low-Pass Mask")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(high_pass_mask[:, :, 0], cmap="gray")
plt.title("High-Pass Mask")
plt.axis("off")
plt.tight_layout()
plt.show()
```

**OUTPUT-**



#QUESTION AND ANSWER

1. What is the Fourier Transform? Why is it important in digital image processing?

Answer: The Fourier Transform converts an image from the spatial domain into the frequency domain. It represents the image using different frequency components. It is important for image enhancement, filtering, restoration, and analysis.

2. Differentiate between the spatial domain and the frequency domain.

Answer: The spatial domain processes image pixels directly, while the frequency domain processes the frequency components of an image. Spatial methods modify pixel values; frequency methods modify the spectrum.

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

Answer: The DFT converts a digital image into its frequency representation. It helps identify low- and high-frequency components and enables efficient frequency-domain filtering.

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

Answer: Shifting places the zero-frequency component at the center, making the spectrum easier to visualize and analyze. Low frequencies appear near the center, while high frequencies appear farther away.

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Answer: Low-pass filters allow low frequencies and reduce high frequencies, helping to smooth images and remove noise. High-pass filters allow high frequencies and reduce low frequencies, helping to sharpen images and detect edges.

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

Answer: The IDFT converts the processed frequency-domain image back into the spatial domain. It reconstructs the image after filtering or other frequency-domain operations.

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

Answer: It allows selective control of frequency components and is effective for operations such as smoothing, sharpening, and periodic noise removal. It is especially useful when the desired enhancement is easier to describe in terms of frequencies.

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Answer:Medical image analysis (MRI and CT). Satellite and remote-sensing image processing. Industrial defect detection. Face recognition and biometric analysis.

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Answer: Spatial filtering is simple and efficient for small kernels and local operations. Frequency filtering is often more efficient for large kernels because convolution can be performed using multiplication in the frequency domain. Spatial filtering is common for local enhancement, while frequency filtering is useful for large-scale smoothing, sharpening, and periodic noise removal.

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

Answer: It helps remove noise and blur by modifying unwanted frequency components. It also enhances edges and textures, making important features easier to detect and extract.